

Homework 3: Solutions

Latent Factors, Embeddings, and Attention

Advanced Methods in Deep Learning — Universidad del Pacífico

Contents

1	SVD and Matrix Factorization	2
2	Embeddings and Word2Vec	4
3	Attention and Self-Attention	7

1 SVD and Matrix Factorization

Problem 1 — Trace Cyclic Identity [4 pts]

(a) Prove $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$ for $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times m}$.

Expand using the definition $\text{tr}(\mathbf{M}) = \sum_i M_{ii}$:

$$\text{tr}(\mathbf{AB}) = \sum_{i=1}^m (\mathbf{AB})_{ii} = \sum_{i=1}^m \sum_{j=1}^n A_{ij} B_{ji} = \sum_{j=1}^n \sum_{i=1}^m B_{ji} A_{ij} = \sum_{j=1}^n (\mathbf{BA})_{jj} = \text{tr}(\mathbf{BA}).$$

The interchange of summation order is justified because the double sum is finite. $\square$

(b) Generalize to $\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB})$.

Let $\mathbf{M} = \mathbf{BC}$. By part (a):

$$\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{AM}) = \text{tr}(\mathbf{MA}) = \text{tr}(\mathbf{BCA}).$$

For the second equality, let $\mathbf{N} = \mathbf{CA}$. By part (a):

$$\text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{BN}) = \text{tr}(\mathbf{NB}) = \text{tr}(\mathbf{CAB}). \quad \square$$

Problem 2 — SVD $\leftrightarrow$ PCA Eigenvalues [4 pts]

(a) Show $\mathbf{XX}^\top = \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^\top$.

Given $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ with $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$:

$$\mathbf{XX}^\top = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top)(\mathbf{V}\mathbf{\Sigma}^\top\mathbf{U}^\top) = \mathbf{U}\mathbf{\Sigma}(\mathbf{V}^\top\mathbf{V})\mathbf{\Sigma}^\top\mathbf{U}^\top = \mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^\top\mathbf{U}^\top = \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^\top.$$

Since $\mathbf{U}$ is orthogonal this is a spectral decomposition, so the eigenvalues of $\mathbf{XX}^\top$ are exactly the diagonal entries σ_d^2 .

(b) Argue that the eigenvalues of $\mathbf{XX}^\top$ are non-negative (no SVD needed).

$\mathbf{XX}^\top$ is a Gram matrix (positive semi-definite). For any $\mathbf{z} \in \mathbb{R}^D$:

$$\mathbf{z}^\top(\mathbf{XX}^\top)\mathbf{z} = \|\mathbf{X}^\top\mathbf{z}\|^2 \geq 0.$$

Since the quadratic form is non-negative for all $\mathbf{z}$, all eigenvalues are ≥ 0 . $\square$

Problem 3 — Truncated SVD Storage [3 pts]

(a) Storage count for rank- k SVD of $\mathbf{X} \in \mathbb{R}^{N \times N}$:

$$\underbrace{Nk}_{\mathbf{U}_k} + \underbrace{k}_{\mathbf{\Sigma}_k} + \underbrace{Nk}_{\mathbf{V}_k^\top} = 2Nk + k = k(2N + 1) \text{ real numbers.}$$

(b) When is the truncated SVD strictly smaller than the dense $N \times N$ matrix?

We need $k(2N + 1) < N^2$, i.e.

$$k < \frac{N^2}{2N + 1} \approx \frac{N}{2} \text{ for large } N.$$

Precisely, any integer k satisfying $k < N^2/(2N + 1)$. For example, if $N = 100$, we need $k \leq 49$.

Problem 4 — Matrix Factorization Gradients [5 pts]

Given $L(\mathbf{U}, \mathbf{V}) = \frac{\lambda}{2}(\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2) + \frac{1}{2} \sum_{(i,j) \in S} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2$.

Define the residual $e_{ij} = y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j$.

Gradient w.r.t. $\mathbf{u}_i$: Only terms involving row i contribute:

$$\frac{\partial L}{\partial \mathbf{u}_i} = \lambda \mathbf{u}_i + \sum_{j:(i,j) \in S} \frac{\partial}{\partial \mathbf{u}_i} \frac{1}{2} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2 = \lambda \mathbf{u}_i - \sum_{j:(i,j) \in S} e_{ij} \mathbf{v}_j.$$

Gradient w.r.t. $\mathbf{v}_j$: Symmetrically, only terms involving column j :

$$\frac{\partial L}{\partial \mathbf{v}_j} = \lambda \mathbf{v}_j - \sum_{i:(i,j) \in S} e_{ij} \mathbf{u}_i.$$

Problem 5 — ALS Closed Form [5 pts]

Fix $\mathbf{V}$ and minimize L over $\mathbf{u}_i$.

Setting $\partial L / \partial \mathbf{u}_i = 0$ from Problem 4:

$$\lambda \mathbf{u}_i = \sum_{j \in S_i} e_{ij} \mathbf{v}_j = \sum_{j \in S_i} (y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j) \mathbf{v}_j = \sum_{j \in S_i} y_{ij} \mathbf{v}_j - \left(\sum_{j \in S_i} \mathbf{v}_j \mathbf{v}_j^\top \right) \mathbf{u}_i.$$

where $S_i = \{j : (i, j) \in S\}$. Rearranging:

$$\left(\lambda \mathbf{I} + \sum_{j \in S_i} \mathbf{v}_j \mathbf{v}_j^\top \right) \mathbf{u}_i = \sum_{j \in S_i} y_{ij} \mathbf{v}_j.$$

Let $\mathbf{V}_i \in \mathbb{R}^{|S_i| \times K}$ be the matrix whose rows are $\mathbf{v}_j^\top$ for $j \in S_i$, and $\mathbf{y}_i$ the corresponding observations. Then in matrix form:

$$\boxed{\mathbf{u}_i = (\lambda \mathbf{I} + \mathbf{V}_i^\top \mathbf{V}_i)^{-1} \mathbf{V}_i^\top \mathbf{y}_i.}$$

Problem 6 — Panel Data with One Missing Potential Outcome [9 pts]

$\mathbf{Y} = \begin{pmatrix} 2 & 4 & 6 \\ 1 & 2 & 3 \\ 3 & ? & 9 \\ 4 & 8 & 12 \end{pmatrix}$, column factors fixed at $\mathbf{v} = (1, 2, 3)^\top$, rank $K = 1$, $\lambda = 0$.

(a) ALS update: $\hat{u}_i = \frac{\sum_{j \in S_i} y_{ij} v_j}{\sum_{j \in S_i} v_j^2}$.

With $K = 1$ and $\lambda = 0$ the ALS formula reduces to a scalar ratio.

Row 1 (all 3 entries observed):

$$\hat{u}_1 = \frac{2(1) + 4(2) + 6(3)}{1^2 + 2^2 + 3^2} = \frac{2 + 8 + 18}{14} = \frac{28}{14} = 2.$$

Row 2 (all 3 observed):

$$\hat{u}_2 = \frac{1(1) + 2(2) + 3(3)}{14} = \frac{14}{14} = 1.$$

Row 3 (entries for $j = 1, 3$ observed; $j = 2$ is missing):

$$\hat{u}_3 = \frac{3(1) + 9(3)}{1^2 + 3^2} = \frac{3 + 27}{10} = \frac{30}{10} = 3.$$

Row 4 (all 3 observed):

$$\hat{u}_4 = \frac{4(1) + 8(2) + 12(3)}{14} = \frac{4 + 16 + 36}{14} = \frac{56}{14} = 4.$$

(b) Predict the counterfactual $\hat{Y}_{3,2}(0)$.

$$\hat{Y}_{3,2}(0) = \hat{u}_3 \cdot v_2 = 3 \times 2 = \mathbf{6}.$$

(c) Treatment effect estimate.

$$\hat{\tau}_{3,2} = Y_{3,2}(1) - \hat{Y}_{3,2}(0) = 8 - 6 = \mathbf{2}.$$

(d) Connection to synthetic control.

Synthetic control constructs the counterfactual for a treated unit as a *fixed, hand-crafted* convex combination of control units, relying on a researcher-chosen donor pool and pre-treatment fit. The matrix completion approach is more general: it imputes missing potential outcomes from a *data-driven* low-rank factor structure that simultaneously learns latent time factors and unit loadings from all observed entries, without constraining the loadings to sum to one or restricting which units serve as controls. As the number of time periods and units grows, the factor model recovers the counterfactual consistently, whereas synthetic control breaks down when no convex combination of donors fits the pre-treatment trajectory.

2 Embeddings and Word2Vec

Problem 7 — Softmax Gradient [5 pts]

Let $s_w = \mathbf{v}'_w \mathbf{v}_{w_I}$. Then:

$$\log P(w_O | w_I) = s_{w_O} - \log \sum_{w=1}^W e^{s_w}.$$

Taking the gradient with respect to $\mathbf{v}_{w_I}$:

$$\begin{aligned} \nabla_{\mathbf{v}_{w_I}} \log P(w_O | w_I) &= \mathbf{v}'_{w_O} - \frac{\sum_w e^{s_w} \mathbf{v}'_w}{\sum_w e^{s_w}} \\ &= \boxed{\mathbf{v}'_{w_O} - \sum_{w=1}^W P(w | w_I) \mathbf{v}'_w}. \end{aligned}$$

Interpretation: The gradient equals the observed output embedding $\mathbf{v}'_{w_O}$ minus the model's expected output embedding $\sum_w P(w | w_I) \mathbf{v}'_w$; it nudges the input embedding $\mathbf{v}_{w_I}$ to become more similar to the true context and less similar to all other context words in proportion to their current predicted probability.

Problem 8 — Time Complexity of Full Softmax [3 pts]

For one pair (w_I, w_O) :

- Computing $s_w = \mathbf{v}_w'^\top \mathbf{v}_{w_I}$ for all W words: $O(WD)$.
- Softmax normalization: $O(W)$.
- Computing $\sum_w P(w|w_I) \mathbf{v}_w'$: $O(WD)$.

Total: $O(WD)$. For typical vocabularies ($W \approx 10^5$, $D \approx 300$) this is 3×10^7 operations per pair — the bottleneck that motivates hierarchical softmax and negative sampling.

Problem 9 — Huffman Tree vs Balanced Binary Tree [9 pts]

Words sorted by frequency (ascending): *of*(4), *you*(4), *devil*(5), *queen*(6), *know*(7), *way*(9), *do*(18), *the*(20). Total: 73.

(a) **Huffman tree construction:**

Step	Action
1	Merge <i>of</i> (4) + <i>you</i> (4) $\rightarrow$ N1(8)
2	Merge <i>devil</i> (5) + <i>queen</i> (6) $\rightarrow$ N2(11)
3	Merge <i>know</i> (7) + N1(8) $\rightarrow$ N3(15)
4	Merge <i>way</i> (9) + N2(11) $\rightarrow$ N4(20)
5	Merge N3(15) + <i>do</i> (18) $\rightarrow$ N5(33)
6	Merge <i>the</i> (20) + N4(20) $\rightarrow$ N6(40)
7	Merge N5(33) + N6(40) $\rightarrow$ root(73)

Resulting path lengths:

Word	Count	Path length L_w
<i>the</i>	20	2
<i>do</i>	18	2
<i>way</i>	9	3
<i>know</i>	7	3
<i>of</i>	4	4
<i>you</i>	4	4
<i>devil</i>	5	4
<i>queen</i>	6	4

(b) **Balanced binary tree (depth 3):**

All 8 words are leaves at depth 3, so $L_w = 3$ for every word.

(c) **Expected path lengths:**

Huffman:

$$\mathbb{E}[L] = \frac{20(2) + 18(2) + 9(3) + 7(3) + 4(4) + 4(4) + 5(4) + 6(4)}{73} = \frac{40 + 36 + 27 + 21 + 16 + 16 + 20 + 24}{73} = \frac{200}{73} \approx 2.74$$

Balanced:

$$\mathbb{E}[L] = 3.$$

Huffman is shorter ($2.74 < 3.00$).

(d) One-sentence explanation:

Huffman assigns shorter codes to the most frequent words (*the, do*) which dominate the expected cost, so even though rare words get longer codes, the frequency-weighted average is lower than a balanced tree that wastes short codes on rare words.

Problem 10 — Negative Sampling Gradient [5 pts]

$$\mathcal{L} = -\log \sigma(\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}) - \sum_{i=1}^k \log \sigma(-\mathbf{v}'_{\tilde{w}_i}{}^\top \mathbf{v}_{w_I}).$$

(a) Derive $\nabla_{\mathbf{v}_{w_I}} \mathcal{L}$.

Using $\frac{d}{dz} \log \sigma(z) = 1 - \sigma(z) = \sigma(-z)$:

Positive term:

$$\frac{\partial}{\partial \mathbf{v}_{w_I}} [-\log \sigma(\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I})] = -\sigma(-\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}) \mathbf{v}'_{w_O} = (\sigma(\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}) - 1) \mathbf{v}'_{w_O}.$$

Each negative term (let $z_i = -\mathbf{v}'_{\tilde{w}_i}{}^\top \mathbf{v}_{w_I}$):

$$\frac{\partial}{\partial \mathbf{v}_{w_I}} [-\log \sigma(z_i)] = -\sigma(-z_i) \cdot \frac{\partial z_i}{\partial \mathbf{v}_{w_I}} = -\sigma(\mathbf{v}'_{\tilde{w}_i}{}^\top \mathbf{v}_{w_I}) \cdot (-\mathbf{v}'_{\tilde{w}_i}) = \sigma(\mathbf{v}'_{\tilde{w}_i}{}^\top \mathbf{v}_{w_I}) \mathbf{v}'_{\tilde{w}_i}.$$

Total:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = (\sigma(\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}) - 1) \mathbf{v}'_{w_O} + \sum_{i=1}^k \sigma(\mathbf{v}'_{\tilde{w}_i}{}^\top \mathbf{v}_{w_I}) \mathbf{v}'_{\tilde{w}_i}.$$

(b) Compare to full-softmax gradient.

Full softmax: $\nabla = \mathbf{v}'_{w_O} - \sum_{w=1}^W P(w|w_I) \mathbf{v}'_w$ — requires summing over *all* W output embeddings.

Negative sampling: the sum $\sum_w P(w|w_I) \mathbf{v}'_w$ is replaced by a sum over only $k+1$ terms: the sigmoid-weighted positive sample plus k sigmoid-weighted noise samples. The role of the full probability distribution is approximated by binary classifiers distinguishing data from noise.

(c) Per-pair time complexity.

$O(kD)$ — only k dot products of dimension D , versus $O(WD)$ for full softmax.

Problem 11 — LLE Numerical Example [8 pts]

Points: $\mathbf{x}_1 = (0, 0)$, $\mathbf{x}_2 = (1, 1)$, $\mathbf{x}_3 = (2, 2)$. $B = 2$, $K = 1$.

(a) Weights for $\mathbf{x}_2$, neighbors $\{1, 3\}$, subject to $W_{2,1} + W_{2,3} = 1$.

We minimize $\|\mathbf{x}_2 - W_{2,1}\mathbf{x}_1 - W_{2,3}\mathbf{x}_3\|^2$ subject to $W_{2,1} + W_{2,3} = 1$.

Substitute $W_{2,3} = 1 - W_{2,1}$ and compute:

$$\mathbf{x}_2 - W_{2,1}\mathbf{x}_1 - (1 - W_{2,1})\mathbf{x}_3 = (1, 1) - W_{2,1}(0, 0) - (1 - W_{2,1})(2, 2) = (1, 1) - (2 - 2W_{2,1}, 2 - 2W_{2,1}) = (2W_{2,1} - 1, 2W_{2,1} - 1).$$

Squared norm: $2(2W_{2,1} - 1)^2$. Setting derivative to zero: $2W_{2,1} - 1 = 0$, so

$$W_{2,1} = W_{2,3} = \frac{1}{2}.$$

Geometrically, $\mathbf{x}_2$ is the midpoint of $\mathbf{x}_1$ and $\mathbf{x}_3$, giving exact reconstruction with zero residual.

(b) Low-dim coordinates satisfying $u_2 = \frac{1}{2}u_1 + \frac{1}{2}u_3$, $\sum_i u_i = 0$, $\frac{1}{3}\sum_i u_i^2 = 1$.

From the reconstruction constraint: $2u_2 = u_1 + u_3$.

From centering: $u_1 + u_2 + u_3 = 0 \Rightarrow u_1 + u_3 = -u_2$.

Substituting: $2u_2 = -u_2 \Rightarrow \boxed{u_2 = 0}$.

Then $u_3 = -u_1$. Unit variance: $\frac{1}{3}(u_1^2 + 0 + u_1^2) = 1 \Rightarrow u_1^2 = \frac{3}{2} \Rightarrow u_1 = \sqrt{\frac{3}{2}}$.

To preserve the ordering $\mathbf{x}_1 < \mathbf{x}_2 < \mathbf{x}_3$, take:

$$\boxed{u_1 = -\sqrt{\frac{3}{2}} \approx -1.225, \quad u_2 = 0, \quad u_3 = \sqrt{\frac{3}{2}} \approx 1.225.}$$

(c) Verification.

Ordering: $u_1 < u_2 < u_3$ matches the 1D ordering $\mathbf{x}_1 \rightarrow \mathbf{x}_2 \rightarrow \mathbf{x}_3$. ✓

Local reconstruction: $\frac{1}{2}u_1 + \frac{1}{2}u_3 = \frac{1}{2}(-\sqrt{3/2}) + \frac{1}{2}(\sqrt{3/2}) = 0 = u_2$. ✓

3 Attention and Self-Attention

Problem 12 — Scaled Dot-Product Attention: Shapes [4 pts]

Input $\mathbf{x} \in \mathbb{R}^{T \times D}$; key dim d_k , value dim d_v .

(a) Weight matrix shapes:

$$\mathbf{W}_Q \in \mathbb{R}^{d_k \times D}, \quad \mathbf{W}_K \in \mathbb{R}^{d_k \times D}, \quad \mathbf{W}_V \in \mathbb{R}^{d_v \times D}.$$

(Convention: $\mathbf{Q} = \mathbf{x}\mathbf{W}_Q^\top$ so the projection maps $D \rightarrow d_k$.)

(b) Derived shapes:

$$\mathbf{Q} \in \mathbb{R}^{T \times d_k}, \quad \mathbf{K} \in \mathbb{R}^{T \times d_k}, \quad \mathbf{V} \in \mathbb{R}^{T \times d_v}, \quad \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{T \times T}.$$

(c) Output shape: $T \times d_v$ (softmax weights are $T \times T$, applied to $\mathbf{V} \in \mathbb{R}^{T \times d_v}$).

Problem 13 — Why $\sqrt{d_k}$? A CLT Argument [8 pts]

$q_d, k_d \stackrel{i.i.d.}{\sim} \mathcal{N}(0, 1)$, $\mathbf{q} \perp \mathbf{k}$, score $s = \mathbf{q}^\top \mathbf{k} = \sum_{d=1}^{d_k} q_d k_d$.

(a) $\mathbb{E}[s]$:

$$\mathbb{E}[s] = \sum_{d=1}^{d_k} \mathbb{E}[q_d] \mathbb{E}[k_d] = 0 \text{ (independence, zero means).}$$

(b) $\text{Var}(s)$:

Since q_d, k_d are independent, $\text{Var}(q_d k_d) = \mathbb{E}[q_d^2 k_d^2] - (\mathbb{E}[q_d k_d])^2 = \mathbb{E}[q_d^2] \mathbb{E}[k_d^2] - 0 = 1$. The d_k terms are independent, so $\text{Var}(s) = d_k$.

(c) Distribution for large d_k :

The score $s = \sum_{d=1}^{d_k} q_d k_d$ is a sum of d_k i.i.d. mean-zero, finite-variance terms. By the CLT:

$$s \xrightarrow{d} \mathcal{N}(0, d_k) \quad \text{as } d_k \rightarrow \infty.$$

(d) Why dividing by $\sqrt{d_k}$ fixes softmax saturation:

After scaling, $s/\sqrt{d_k}$ has variance $d_k/d_k = 1$, i.e. $s/\sqrt{d_k} \sim \mathcal{N}(0,1)$ approximately. Without scaling, when d_k is large the scores s have std $\sqrt{d_k} \gg 1$, so the softmax input vector has entries differing by many standard deviations. The softmax then collapses to a near-one-hot vector (the largest score dominates), gradients vanish through that position, and learning stalls. Dividing by $\sqrt{d_k}$ keeps all scores at $O(1)$ magnitude regardless of d_k , maintaining a soft, gradient-friendly attention distribution.

Problem 14 — Multi-Head Attention Shapes [4 pts]

For head i with input $\mathbf{x} \in \mathbb{R}^{T \times D}$, d_k key dim, d_v value dim, h heads:

$$\mathbf{W}_Q^i \in \mathbb{R}^{d_k \times D}, \quad \mathbf{W}_K^i \in \mathbb{R}^{d_k \times D}, \quad \mathbf{W}_V^i \in \mathbb{R}^{d_v \times D}.$$

After concatenation, $[\mathbf{O}^1 \dots \mathbf{O}^h] \in \mathbb{R}^{T \times (h \cdot d_v)}$. The output projection maps this to dimension D :

$$\mathbf{W}_O \in \mathbb{R}^{D \times (h \cdot d_v)}.$$

In the standard setting $d_k = d_v = D/h$, so $h \cdot d_v = D$ and $\mathbf{W}_O \in \mathbb{R}^{D \times D}$.

Problem 15 — Numerical Attention Example [5 pts]

$$T = 2, d_k = d_v = 2; \mathbf{Q} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \mathbf{K} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \mathbf{V} = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}.$$

Step 1. Compute score matrix $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$:

$$\mathbf{Q}\mathbf{K}^\top = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}^\top = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

(Note: $\mathbf{K}^\top = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ since $\mathbf{K}_{dj}^\top = K_{jd}$.)

Wait, let me redo: $\mathbf{K} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ so $\mathbf{K}^\top = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$.

$$\mathbf{Q}\mathbf{K}^\top = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 1 & 1 \cdot 1 + 0 \cdot (-1) \\ 0 \cdot 1 + 1 \cdot 1 & 0 \cdot 1 + 1 \cdot (-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Dividing by $\sqrt{2}$: $\mathbf{S} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$.

Step 2. Apply row-wise softmax. Let $\alpha = e^{1/\sqrt{2}} \approx 2.028$, $\beta = e^{-1/\sqrt{2}} \approx 0.493$.

Row 1: scores $(1/\sqrt{2}, 1/\sqrt{2}) \Rightarrow$ weights $= (0.5, 0.5)$.

Row 2: scores $(1/\sqrt{2}, -1/\sqrt{2}) \Rightarrow$ weights $= \left(\frac{\alpha}{\alpha + \beta}, \frac{\beta}{\alpha + \beta} \right) \approx (0.805, 0.195)$.

Step 3. Compute output $\mathbf{O} = \mathbf{A}\mathbf{V}$:

$$\mathbf{O}_{1,\cdot} = 0.5 \begin{pmatrix} 2 \\ 0 \end{pmatrix} + 0.5 \begin{pmatrix} 0 \\ 4 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

$$\mathbf{O}_{2,\cdot} = \frac{\alpha}{\alpha + \beta} \begin{pmatrix} 2 \\ 0 \end{pmatrix} + \frac{\beta}{\alpha + \beta} \begin{pmatrix} 0 \\ 4 \end{pmatrix} = \begin{pmatrix} 2\alpha/(\alpha + \beta) \\ 4\beta/(\alpha + \beta) \end{pmatrix} \approx \begin{pmatrix} 1.610 \\ 0.779 \end{pmatrix}.$$

$$\mathbf{O} \approx \begin{pmatrix} 1.000 & 2.000 \\ 1.610 & 0.779 \end{pmatrix}.$$

Problem 16 — Causal Mask: Explicit Example [4 pts]

$T = 4$. Unmasked scores $\mathbf{S}_{ij} = i - j$ (0-indexed row/col) rearranged as given.

(a) Score matrix after causal mask:

$$\tilde{\mathbf{S}} = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 1 & 0 & -\infty & -\infty \\ 2 & 1 & 0 & -\infty \\ 3 & 2 & 1 & 0 \end{pmatrix}.$$

(b) Softmax for row $i = 2$ (0-indexed $i = 1$): scores $(1, 0, -\infty, -\infty)$.

The $-\infty$ entries map to $e^{-\infty} = 0$. The active scores are $(1, 0)$:

$$\text{softmax}(1, 0) = \left(\frac{e^1}{e^1 + e^0}, \frac{e^0}{e^1 + e^0} \right) = \left(\frac{e}{e + 1}, \frac{1}{e + 1} \right) \approx (0.731, 0.269).$$

Full attention row: $\left(\frac{e}{e+1}, \frac{1}{e+1}, 0, 0 \right) \approx (0.731, 0.269, 0, 0)$.